Competition Study Guide

Graph Learning-to-Rank for TPU Runtime Prediction · for a CS undergraduate

Google – Fast or Slow? Predict AI Model Runtime · Generated 2026-06-18

0. How to use this guide

This guide is the theory companion to the project workflow. It is written for a **computer-science undergraduate** who has completed the standard syllabus — calculus, linear algebra, probability, data structures and algorithms, and one introductory machine learning course — but who has **not** yet studied graph neural networks, learning-to-rank, PyTorch, or TPU compilation. Everything specific to those topics is built up from scratch, with the mathematics included rather than hidden.

Assumed background. You should be comfortable with: vectors and matrices, matrix multiplication and transposes; gradients and the chain rule; probability distributions, expectation, and maximum likelihood; the idea of training a model by minimising a loss with gradient descent; and overfitting / train–validation–test discipline.

How to read it. Sections 1–3 frame the *problem* and the two mathematical languages it is written in (ranking and graphs). Sections 4–6 build the *model* (neural networks, then graph neural networks, then how to scale them). Sections 7–9 connect everything to *this competition* and to sound experimental practice. Section 10 is a glossary, a reference list, and a suggested study order mapped onto the workflow phases.

Symbols: scalars a, vectors $\mathbf{x}$, matrices A, sets $\mathcal{V}$. $\sigma(\cdot)$ is a nonlinearity (often sigmoid or ReLU, disambiguated in context). $\|$ denotes vector concatenation.

---

1. The domain and the problem

1.1 What is actually being optimised

Modern machine-learning models (a BERT language model, a ResNet image classifier) are not executed as Python line by line. A compiler first turns the model into a **computation graph** and then decides *how* to run each operation on the target hardware. For Google's **TPUs** (Tensor Processing Units — accelerators specialised for large tensor algebra), this compiler is **XLA** (Accelerated Linear Algebra). XLA represents the program as **HLO** (High-Level Optimizer) operations: a directed graph in which each node is an operation (matrix multiply, convolution, add, reshape, …) and each edge is a tensor flowing from one operation to the next.

A single computation graph can be compiled in **many different ways**, and these choices have a large effect on how fast the program runs. Two families of choices appear in this competition:

- **Layout configurations.** How multi-dimensional tensors are physically arranged in memory (the order of their dimensions). A good layout for one operation may force an expensive re-arrangement before the next, so layout is a *global*, graph-wide decision.
- **Tile-size configurations.** How a large operation is broken into smaller blocks ("tiles") that fit the hardware's fast memory. This is a more *local* decision attached to individual fused operations.

1.2 Why predict runtime instead of measuring it

The compiler must search a huge space of configurations to find a fast one. Actually **measuring** the runtime of each candidate on real hardware is far too slow and expensive to do for every option during compilation. Instead, compilers use a **cost model**: a function that *predicts* the runtime (or at least the relative ranking) of a configuration from features of the graph and the configuration, cheaply and without running it. A better cost model lets the compiler's search find faster programs, which speeds up real workloads across data centres.

This competition asks you to learn that cost model from data. You are given a large dataset of (graph, configuration, measured runtime) triples and must build a model that predicts which configurations are fast.

1.3 The key reframing: ranking, not regression

Here is the single most important conceptual point. The compiler does **not** need to know the *absolute* runtime of a configuration in microseconds. It needs to know **which configuration is fastest** — that is, the *relative order*. Two cost models that disagree about absolute runtimes but agree about the ordering are equally useful.

Consequently the competition is scored on **ranking quality**, and we should train the model to rank, not to predict exact numbers. Predicting absolute runtimes (regression) wastes capacity on a harder problem than

we need to solve, and it is sensitive to scale differences between graphs that are irrelevant to the task. This is why Section 2 develops **learning-to-rank** in detail; it is the mathematical heart of the competition.

1.4 The search space and why it is hard

For a given graph, the number of candidate configurations ranges from a few hundred (tile) to tens of thousands (layout). The graphs themselves range from a dozen nodes to tens of thousands of nodes. So the model must:

1. read a **variable-size graph**,
2. read **per-configuration** features (which, for layout, live only on a subset of nodes),
3. and output **one score per configuration** such that sorting by score reproduces the true fastest-to-slowest order.

That combination — graph-structured input, set-valued output, ranking objective, extreme size variation — is what makes this a genuinely hard and interesting problem.

---

2. Learning to rank, formally

2.1 Problem set-up

In learning-to-rank we are given a collection of **lists**. In our case one list = one graph with its set of candidate configurations. For a list with n items, each item i has a feature representation and a **relevance label** y_i — here, the (negative) measured runtime, so that "more relevant" means "faster". The model produces a **score** $s_i = f(\text{item}_i)$, and we sort items by descending score to get the predicted ranking. The goal is for the predicted order to match the order induced by the labels y_i.

Three broad strategies exist, distinguished by how much of the list structure the loss sees.

2.2 Pointwise approaches (and why we avoid them)

A **pointwise** method treats each item independently and regresses the score toward the label, e.g. mean-squared error $\sum_i (s_i - y_i)^2$. This is exactly the regression framing we argued against: it forces the model to match absolute values, it lets one graph's runtime scale dominate the loss, and it does not directly optimise ordering. Pointwise methods are a useful *sanity baseline* but rarely competitive for ranking.

2.3 Pairwise approaches

A **pairwise** method looks at *pairs* of items and tries to get each pair in the right order. Define, for a pair (i, j) where $y_i > y_j$ (item i should rank above j), the score difference $s_i - s_j$. We want this difference to be positive and large.

RankNet turns this into a probability with the logistic function $\sigma(z) = 1/(1 + e^{-z})$:

$$P(i \succ j) = \sigma(s_i - s_j) = \frac{1}{1 + e^{-(s_i - s_j)}},$$

and minimises the binary cross-entropy against the target $\bar{P}_{ij} = 1$ when $y_i > y_j$:

$$\mathcal{L}_{\text{RankNet}} = \sum_{(i,j):\, y_i > y_j} -\log \sigma(s_i - s_j).$$

A simpler, very robust alternative is the **pairwise hinge (margin) loss**, which asks the correct item to beat the other by at least a margin $m > 0$:

$$\mathcal{L}_{\text{hinge}} = \sum_{(i,j):\, y_i > y_j} \max\bigl(0,\; m - (s_i - s_j)\bigr).$$

Pairwise losses directly optimise *orderings* and are far better aligned with the metric than pointwise losses. Their main cost is that a list of n items has $O(n^2)$ pairs; in practice we **sample** a fixed number of configurations per graph to form each list (Section 8).

2.4 Listwise approaches and ListMLE

A **listwise** method scores the *entire ordering* at once. The most important example for this competition is **ListMLE**, which is built on the **Plackett–Luce** model of rankings.

The Plackett–Luce model. Suppose each item i has a positive "strength" $\exp(s_i)$. Generate a ranking by repeatedly drawing the next item with probability proportional to its remaining strength. If π is a permutation with $\pi(1)$ ranked first, $\pi(2)$ second, and so on, then

$$P(\pi \mid \mathbf{s}) \;=\; \prod_{i=1}^{n} \frac{\exp\left(s_{\pi(i)}\right)}{\sum_{k=i}^{n} \exp\left(s_{\pi(k)}\right)}.$$

Read the i-th factor as: "given that ranks $1..i-1$ are already filled, the probability that item $\pi(i)$ is chosen next is its strength divided by the total strength of all items not yet placed."

ListMLE loss. Let $\pi^{\backslash *}$ be the *ground-truth* ordering (sort items by label). ListMLE is the **negative log-likelihood** of that ground-truth ordering under Plackett–Luce:

$$\mathcal{L}_{\text{ListMLE}} \;=\; -\log P(\pi^{\backslash *} \mid \mathbf{s}) \;=\; -\sum_{i=1}^{n} \left[s_{\pi^{\backslash *}(i)} \;-\; \log \sum_{k=i}^{n} \exp\left(s_{\pi^{\backslash *}(k)}\right) \right].$$

Minimising it pushes the score of the true-first item above all others, then the true-second item above all the remaining, and so on — exactly the structure we want. The inner $\log \sum \exp$ terms can be computed in one backward pass from the end of the list, so the loss is $O(n)$ given the sorted scores. ListMLE is the loss used in the official starter notebook and is a strong default for this task. (A closely related listwise loss, **ListNet**, minimises the cross-entropy between the top-one probability distributions $\mathrm{softmax}(\mathbf{s})$ and $\mathrm{softmax}(\mathbf{y})$.)

2.5 Evaluation metrics

We optimise a smooth loss, but we are *judged* on a ranking metric. Know both.

Concordant and discordant pairs. For two items with distinct labels, the model orders the pair **concordantly** if its scores agree with the labels ($y_i > y_j$ and $s_i > s_j$), and **discordantly** otherwise.

Kendall's τ. With C concordant and D discordant pairs out of $\binom{n}{2}$ total,

$$\tau \;=\; \frac{C - D}{\binom{n}{2}} \;\in\; [-1, 1],$$

where $\tau = 1$ is a perfect ranking and $\tau = -1$ a perfectly reversed one.

Ordered-Pair Accuracy (OPA). The fraction of pairs ordered correctly,

$$\mathrm{OPA} \;=\; \frac{C}{C + D},$$

which (with no ties) relates to Kendall's τ by $\mathrm{OPA} = (\tau + 1)/2$. OPA is the quantity tracked during training in the starter notebook and is an intuitive "what fraction of comparisons did I get right" number.

NDCG (Normalised Discounted Cumulative Gain) rewards putting highly relevant items near the top, with a logarithmic position discount:

$$\mathrm{DCG} = \sum_{i=1}^{n} \frac{2^{\,\mathrm{rel}_i} - 1}{\log_2(i+1)}, \qquad \mathrm{NDCG} = \frac{\mathrm{DCG}}{\mathrm{IDCG}},$$

where IDCG is the DCG of the ideal ordering. NDCG matters when only the top of the ranking is used.

The competition's metrics. The two task families are scored differently, reflecting how the predictions would be used:

- **Layout** is scored with a **Kendall-τ-style** measure over the full predicted ordering of configurations — the whole order matters.
- **Tile** is scored with a **top-K slowdown** measure: among the model's top-K predicted configurations (e.g. $K = 5$), take the truly fastest one and compare its runtime to the global best. Writing $r_{g,j}$ for the runtime of configuration j on graph g,

$$\mathrm{slowdown}_g \;=\; \frac{\min_{j \in \text{top-}K(g)} r_{g,j}}{\min_j r_{g,j}} \;-\; 1 \;\geq\; 0,$$

and the score rewards a *small* average slowdown (the model only needs a good configuration near the top, not a perfect full ordering). The overall competition score combines the collections, weighting **layout** most heavily — which is why the workflow prioritises it.

The practical takeaway: **train ListMLE or pairwise; evaluate with OPA/Kendall offline; and remember the tile metric only cares about the top of the list.**

---

3. Graphs, formally

3.1 Definitions

A **directed graph** $G = (\mathcal{V}, \mathcal{E})$ has a node set $\mathcal{V}$ with $|\mathcal{V}| = n$ and an edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ with $|\mathcal{E}| = m$. An edge (u, v) points from source u to target v. Each node i carries a feature vector $\mathbf{x}_i \in \mathbb{R}^d$; stacking them gives the **node feature matrix** $X \in \mathbb{R}^{n \times d}$.

A computation graph is naturally directed: an operation's output tensor *feeds* the next operation. The direction matters — "information flows downstream" — which is why, when we do message passing, we will be careful about edge orientation.

3.2 Adjacency, degree, and the Laplacian

The **adjacency matrix** $A \in \{0, 1\}^{n \times n}$ has $A_{uv} = 1$ iff $(u, v) \in \mathcal{E}$. The **degree matrix** D is diagonal with $D_{ii} = \sum_j A_{ij}$ (the out-degree; for undirected graphs, simply the degree). Left-multiplying features by A **aggregates neighbours**: row i of AX is the sum of the feature vectors of i's neighbours. This single fact — that "multiply by the adjacency = sum over neighbours" — is the engine of every graph neural network.

The (combinatorial) **graph Laplacian** is $L = D - A$, and its **symmetric normalised** form is

$$L_{\text{sym}} = I - D^{-1/2} A D^{-1/2}.$$

The Laplacian is to graphs what the second derivative is to functions: $\mathbf{x}^\top L \mathbf{x} = \frac{1}{2} \sum_{(i,j)} (x_i - x_j)^2$ measures how much a signal varies across edges. Its eigen-decomposition defines a notion of "graph frequency", which is the starting point for **spectral** graph convolutions (Section 5.2).

3.3 Sparse representation (COO)

Real graphs are sparse: $m \ll n^2$. We never store the full $n \times n$ matrix. The standard **coordinate (COO)** format stores two integer arrays, `src` and `dst`, each of length m, listing the endpoints of every edge. (In this dataset the raw `edge_index` is stored as an $[m, 2]$ array — a list of (src, dst) pairs — which most graph libraries expect transposed to $[2, m]$; getting this orientation right is a classic source of silent bugs.) Aggregation is then implemented with a **scatter-add**: for each edge, add the source node's message into the destination node's bucket. This is $O(m)$ time and memory, which is what makes GNNs tractable on huge graphs.

---

4. Neural-network building blocks (framework-agnostic)

This section establishes the deep-learning vocabulary used later, independent of any library. If you have seen logistic regression and the chain rule, you have the prerequisites.

4.1 Tensors and the computation graph of a model

A **tensor** is an n-dimensional array (a scalar is 0-D, a vector 1-D, a matrix 2-D). A neural network is a composition of tensor operations; conceptually it is itself a directed graph whose nodes are operations — do not confuse this *autodiff* graph with the *data* graph the GNN consumes.

4.2 Autodiff and backpropagation

Training minimises a scalar loss $\mathcal{L}(\theta)$ over parameters θ by gradient descent: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$. The gradient is computed by **backpropagation**, which is the chain rule applied in reverse over the operation graph. If $\mathcal{L}$ depends on θ through intermediate u, then

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial u}\,\frac{\partial u}{\partial \theta}.$$

A **forward pass** computes outputs and caches intermediates; a **backward pass** propagates $\partial \mathcal{L}/\partial(\cdot)$ from the loss back to every parameter. Frameworks (PyTorch/TensorFlow) do this automatically — you write the forward pass and get gradients for free. One consequence we use later: wrapping a sub-computation in `stop_gradient` (detach) makes the backward pass treat it as a constant, which is central to Graph Segment Training.

4.3 The multilayer perceptron (MLP)

The basic learnable transformation is an **affine map followed by a nonlinearity**: a layer computes $\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$ with weight matrix W, bias $\mathbf{b}$, and an elementwise nonlinearity such as **ReLU** $\sigma(z) = \max(0, z)$ or **LeakyReLU** $\sigma(z) = \max(\alpha z, z)$. Stacking such layers gives an MLP, a universal function approximator. MLPs appear inside GNNs as the per-node "update" networks.

4.4 Embeddings for categorical inputs

Some inputs are **categorical**, e.g. the **op-code** that says whether a node is a matmul, a convolution, an add, etc. We cannot feed a raw integer ID into a network meaningfully (ID 7 is not "between" 6 and 8). Instead we learn an **embedding table** $E \in \mathbb{R}^{K \times e}$: a lookup that maps each of the K categories to a trainable e-dimensional vector. The op-code embedding is learned jointly with the rest of the model so that operationally similar op-codes end up with similar vectors.

4.5 Normalisation, regularisation, and optimisation

- **Feature normalisation.** Inputs with wildly different scales (tensor sizes can span many orders of magnitude) destabilise training. Standardise to zero mean / unit variance, and apply $\log(1+x)$ to heavy-tailed size features first. Crucially, fit normalisation statistics on the **training split only** to avoid leakage.
- **Regularisation.** L_2 weight decay ($+\lambda\|\theta\|^2$) and **dropout** (randomly zeroing activations during training) combat overfitting.
- **Optimiser. Adam** adapts a per-parameter learning rate from running estimates of the first and second moments of the gradient; it is the default for GNNs. **Gradient clipping** (capping the gradient norm) prevents the occasional exploding update on irregular graphs.

---

5. Graph neural networks, in depth

5.1 The message-passing framework

A **graph neural network** computes node representations by repeatedly mixing each node's vector with those of its neighbours. One layer of **message passing** updates every node i:

$$\mathbf{h}_i^{(l+1)} = \mathrm{UPDATE}\Big(\mathbf{h}_i^{(l)},\ \mathrm{AGG}\big(\{\,\mathrm{MSG}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}) : j \in \mathcal{N}(i)\,\}\big)\Big),$$

where $\mathcal{N}(i)$ are the neighbours of i, **MSG** builds a message along each edge, **AGG** is a permutation-invariant aggregator (sum, mean, or max — it must not depend on the arbitrary order of neighbours), and **UPDATE** combines the aggregate with the node's own state. After L layers, each node has "seen" its L-hop neighbourhood. A **readout** then pools node vectors into a graph-level vector when a single graph-level output is needed.

Almost every GNN is a special case of this template; they differ in the choice of MSG, AGG, and UPDATE.

5.2 From spectral convolutions to the GCN

The **Graph Convolutional Network** (Kipf & Welling, 2017) is the canonical starting point, and it is worth deriving because it explains the ubiquitous "normalised adjacency".

Spectral graph theory defines convolution via the Laplacian's eigenbasis, but using it directly is expensive. GCN makes two approximations: restrict filters to the first-order neighbourhood, and share a single parameter. This collapses a spectral filter to a simple local averaging operation. To keep activations stable, GCN adds **self-loops** ($\tilde{A} = A + I$, so a node includes itself in its own neighbourhood) and uses the **symmetric normalisation** with the corresponding degree matrix $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$:

$$\hat{A} = \tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}.$$

The normalisation rescales each edge by $1/\sqrt{\deg(i)\,\deg(j)}$, preventing high-degree nodes from dominating and keeping the spectral radius bounded (so deep stacks do not blow up). A GCN layer is then

$$H^{(l+1)} = \sigma\big(\hat{A}\,H^{(l)}\,W^{(l)}\big),$$

i.e. **aggregate neighbours ($\hat{A}H$), transform (W), and apply a nonlinearity.** The starter notebook's model is a residual variant of exactly this, built from the implicit adjacency operators.

5.3 GraphSAGE: sampling and concatenation

GraphSAGE (Hamilton et al., 2017) keeps a node's own representation separate from its neighbours' by **concatenating** rather than summing them, and it works with a *sampled* subset of neighbours for scalability:

$$\mathbf{h}_i^{(l+1)} = \sigma\Big(W \cdot \big[\, \mathbf{h}_i^{(l)} \,\|\, \mathrm{AGG}\big(\{\mathbf{h}_j^{(l)} : j \in \mathcal{N}_{\text{sampled}}(i)\}\big) \,\big]\Big).$$

The separation of "self" and "neighbour" channels often helps, and neighbour sampling is one route to scaling (Section 6).

5.4 Graph Attention Networks (GAT)

GAT (Veličković et al., 2018) lets a node weight its neighbours *unequally* via learned attention. For each edge it computes an unnormalised score and then a softmax over a node's incoming edges:

$$e_{ij} = \mathrm{LeakyReLU}\big(\mathbf{a}^\top [\, W\mathbf{h}_i \,\|\, W\mathbf{h}_j \,]\big), \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})},$$

$$\mathbf{h}_i^{(l+1)} = \sigma\Big(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}\, W\mathbf{h}_j \Big).$$

Multiple attention "heads" are run in parallel and concatenated. Attention is useful when only some neighbours matter for runtime (e.g. the operations on a critical path).

5.5 Graph transformers and GPS

Message passing only mixes *local* information per layer, which struggles with long-range dependencies (Section 5.7). **Graph transformers** add global self-attention across all nodes, and the **GPS** framework (Rampášek et al., 2022) *combines* a local message-passing layer with a global attention layer in each block, plus **positional/structural encodings** that tell the attention where each node sits in the graph. GPS-style models are strong on the large TPUGraphs graphs and are worth trying in the solution-quality phase.

5.6 Readout / pooling

To produce a per-graph (or per-configuration) score we pool node vectors with a permutation-invariant **readout** — sum, mean, or max over the relevant nodes, or an attention-weighted pool. In this competition the pooling is *targeted*: for a layout configuration we pool primarily over the **configurable** nodes (those the configuration actually touches), and combine that with a global graph summary.

5.7 Failure modes: over-smoothing and over-squashing

Two phenomena limit naïve deep GNNs:

- **Over-smoothing.** Stack too many message-passing layers and every node's representation converges to the same vector (repeated neighbour-averaging is a diffusion that erases differences). Mitigations: residual/skip connections, jumping-knowledge, and keeping depth modest (often 2–6 layers).
- **Over-squashing.** Information from an exponentially growing receptive field is compressed into a fixed-size vector, so distant signals get "squashed". Global nodes, attention, and GPS-style architectures relieve this.

5.8 Expressive power and the Weisfeiler–Lehman test

How powerful are message-passing GNNs? A classical result (Xu et al., 2019; Morris et al., 2019) shows that standard message-passing GNNs are **at most as discriminative as the 1-dimensional Weisfeiler–Lehman (1-WL) graph-isomorphism test**. The **1-WL** algorithm iteratively refines a colour per node by hashing the multiset of neighbour colours; two graphs it cannot tell apart, a plain GNN cannot tell apart either. The practical lesson is that **raw structure alone has limits**, so injecting informative **node and structural features** (degrees, depths, positional encodings) genuinely increases what the model can represent — motivating the feature engineering in the workflow.

---

6. Scaling GNNs to enormous graphs

Layout graphs reach **tens of thousands of nodes**, and each must be scored under **thousands of configurations**. A full-graph forward and backward pass for every configuration does not fit in memory. Three ideas address this.

6.1 Neighbour sampling

Instead of using all neighbours at each layer, sample a fixed number (GraphSAGE-style). This bounds the receptive field's growth and memory per node, at the cost of some variance.

6.2 Subgraph / mini-batch training

Partition the graph into subgraphs (clusters) and train on one subgraph at a time (Cluster-GCN-style). Each step sees a coherent local region of the graph at a fraction of the memory.

6.3 Graph Segment Training (the key technique here)

Graph Segment Training (GST; the segment-dropout idea used in the TPUGraphs work, NeurIPS 2023) is the method the starter notebook uses and the most important one for layout. The idea:

1. Split the graph's nodes into **segments**.
2. Run a **full-graph forward pass** to get context, but under `stop_gradient` so it costs no backward memory.
3. Keep one (or a few) **segments "live"** and run a second forward pass *only* through those nodes' edges, *with* gradients.
4. Combine: live nodes use the gradient-carrying representation, the rest use the detached full-graph one. Backpropagate only through the live segment.

This decouples **memory** (set by the live-segment size you can afford) from **graph size** (which can be arbitrarily large), making it possible to train on graphs that would otherwise never fit. The number of kept nodes is the central memory knob. In the starter notebook this appears as the `sampled_config` / `sampled_feed` edge sets and the `MAX_KEEP_NODES` constant.

6.4 A note on memory accounting

Backward memory scales roughly with the number of activations you must keep to differentiate — i.e. with **live** nodes × layers × hidden size, not with the full graph. Mixed-precision (16-bit) activations and gradient accumulation further stretch the budget. Inference, needing no gradients, can process the full graph in chunks of configurations.

---

7. The TPUGraphs dataset, in detail

7.1 Collections

The data is split into **five collections** across two task families:

Collection	Family	Graphs	Per-config features	Notes
`tile:xla`	tile	many small	`config_feat [c, 24]`	also has `config_runtime_normalizers`
`layout:xla:random`	layout	large	`node_config_feat [c, nc, 18]`	configs from random search
`layout:xla:default`	layout	large	`node_config_feat [c, nc, 18]`	configs near the compiler default
`layout:nlp:random`	layout	large	`node_config_feat [c, nc, 18]`	NLP-model graphs
`layout:nlp:default`	layout	large	`node_config_feat [c, nc, 18]`	NLP-model graphs

"random" vs "default" describes how the candidate configurations were sampled; the two have different runtime distributions, which is why per-collection models help.

7.2 NPZ schema

Each example is a compressed `.npz` archive of NumPy arrays.

Common to all graphs

Key	Shape	Meaning
`node_feat`	`[n_nodes, 140]`	140 numeric features per operation node (shapes, sizes, attributes)
`node_opcode`	`[n_nodes]`	categorical op-code id per node (→ embedding)
`edge_index`	`[n_edges, 2]`	directed edges as `(src, dst)` pairs (transpose for PyG)
`config_runtime`	`[n_configs]`	measured runtime per configuration (the **label**; int64 µs)

Tile-only

Key	Shape	Meaning
`config_feat`	`[n_configs, 24]`	24 features describing each tile configuration
`config_runtime_normalizers`	`[n_configs]`	reference values used to normalise runtimes

Layout-only

Key	Shape	Meaning
`node_config_ids`	`[n_configurable_nodes]`	which nodes are configurable
`node_config_feat`	`[n_configs, nc, 18]`	18 features **per configurable node**, per configuration
`node_splits`	`[1, n_subgraphs]`	subgraph boundaries

7.3 Tile versus layout — the crucial structural difference

This difference dictates the model's input plumbing:

- In **tile**, a configuration is described by a **single 24-vector** for the whole graph (`config_feat[c]`). Wiring it in is easy: broadcast the graph embedding against each configuration's vector.
- In **layout**, a configuration is described by **one 18-vector per configurable node** (`node_config_feat[c, :, :]`), and only the nodes in `node_config_ids` are configurable. You must **scatter** these per-node config features onto the correct nodes of the graph (and leave non-configurable nodes with a zero/pad), then let message passing spread their influence. The starter notebook realises this with a sparse "config" adjacency that maps the virtual config nodes onto op nodes.

7.4 How a configuration maps to a runtime

For a fixed graph, the node features and topology are constant; what varies across configurations is the configuration features (and, for layout, *where* they attach). The model must therefore produce **one score per configuration** by combining the *shared* graph representation with the *configuration-specific* features. Conceptually: encode the graph once, then condition on each configuration to read out its score.

7.5 Label characteristics and pitfalls

- Runtimes are stored as **int64 microseconds**; treat them only as an **ordering** signal.
- The data has **no NaN/Inf** to clean (verified by inspection), so aggressive "cleaning" is unnecessary — and **clipping** `config_runtime` **is actively harmful** because it can change the order of configurations, corrupting the very labels you train on. Normalise *features*, never reorder *labels*.
- Runtime *scales* differ across graphs; since we rank within a graph, never compare raw runtimes across graphs.

---

8. The end-to-end modelling pipeline

This section assembles everything into the path a single example takes from disk to score.

8.1 From NPZ to a batched graph tensor

1. **Read** the `.npz`; convert arrays to tensors with fixed dtypes.
2. **Build the graph**: transpose `edge_index` to `[2, m]`; decide edge direction (downstream along `feed`; the model may also add the reverse and self-loops as in GCN).
3. **Attach features**: node features (normalised, with `log1p` on size features), the op-code **embedding**, and the configuration features (tile: one vector; layout: scattered per-node).
4. **Sample configurations**: pick a fixed number C of configurations per graph (e.g. 8–32) to form a ranking list for the loss; keep *all* configurations at inference.
5. **Batch**: combine several variable-size graphs into one big disconnected graph (the standard GNN mini-batching trick), tracking which nodes belong to which graph for pooling.

8.2 Model and scoring

Encode the graph with a few message-passing layers (Section 5), pool to a per-configuration representation (targeting configurable nodes for layout), and pass it through a small MLP head to emit a scalar **score per configuration**.

8.3 Loss and optimisation

Compute **ListMLE** (or pairwise hinge) over each graph's sampled configuration list against the runtime-induced true order; optimise with **Adam** + gradient clipping; **early-stop on validation OPA**, not on loss.

8.4 Cross-validation done right

Split **by graph**, never by configuration — all configurations of a graph must fall in the same fold, or the model effectively memorises the graph and the validation score becomes a fantasy. Use grouped K-fold (group = graph), and check that mean validation OPA **tracks** the public leaderboard before trusting any improvement.

8.5 Inference and submission

For each test graph, score **all** its configurations, sort by score, and emit the ranked list of configuration indices. The submission row is `<collection>:<graph_id>,<i0;i1;i2; … >`; concatenate all five collections into one CSV with header `ID,TopConfigs`. Validate the format against the sample submission before uploading.

8.6 Feature-engineering catalogue

Auxiliary signals that frequently help (fed *into* the GNN, not replacing it):

- `log1p` of size/shape node features; per-feature standardisation of the rest.
- Higher-dimensional or better-initialised **op-code embeddings**.
- **Config-feature transforms**: per-dimension normalisation, simple interactions, a small learned encoder for the 18-/24-d vectors.
- **Structural features**: in/out degree, an approximate topological depth, subgraph id from `node_splits`, distance-to-output.
- **Graph-level descriptors**: node/edge counts, op-code histograms.

8.7 Ensembling

Ranking tasks ensemble well by **averaging ranks** (or scores) across models that differ by **seed** and by **architecture** (GCN/SAGE/GAT/GPS). Build per-collection ensembles and verify on CV that each member improves the blend. This is usually the cheapest reliable score gain after a solid single model exists.

---

9. Experimentation methodology and reproducibility

- **One change at a time.** Attribute every score move to a single cause; otherwise you learn nothing from a win or a loss.
- **Config-driven runs.** Every experiment is fully described by a YAML file + a fixed seed, so any result can be regenerated.
- **Log everything.** Keep a running table: config hash, per-collection CV-OPA, LB score, and a one-line note. Your future self needs this.
- **Smoke-test first.** Run each new component on a handful of files in seconds before a full training run; most bugs surface there.
- **Trust CV, but verify against LB.** A reliable, leaderboard-correlated validation scheme is worth more than any single modelling trick.
- **Determinism.** Seed Python, NumPy, and the framework; be aware some GPU ops are non-deterministic and document residual variance.

---

10. Glossary, references, and a study order

10.1 Glossary

- **XLA / HLO** — Google's tensor compiler and its operation-graph IR.
- **TPU** — Tensor Processing Unit, Google's ML accelerator.
- **Configuration** — a compilation choice (a layout, or a set of tile sizes) for a graph.
- **Cost model** — a function predicting (the ranking of) runtimes without executing the code.
- **Learning to rank** — training a model to order items rather than predict absolute values.
- **Pointwise / pairwise / listwise** — loss families seeing one item / item pairs / whole lists.
- **Plackett–Luce / ListMLE** — a probabilistic ranking model and the listwise NLL loss built on it.
- **Kendall's τ / OPA / NDCG** — ranking-quality metrics.
- **GNN / message passing** — a network that updates node vectors from their neighbours.
- **GCN / GraphSAGE / GAT / GPS** — specific GNN architectures.
- **Readout / pooling** — aggregating node vectors into a graph- or configuration-level vector.
- **Over-smoothing / over-squashing** — depth-related GNN failure modes.
- **1-WL test** — the colour-refinement isomorphism test bounding GNN expressivity.
- **Graph Segment Training (GST)** — training huge graphs by backpropagating through a kept segment.
- **Embedding** — a learned vector representation of a categorical input (e.g. op-code).
- **COO / `edge_index`** — sparse edge-list representation of a graph.

10.2 References

- **TPUGraphs / Graph Segment Training** — Phothilimthana et al., *TpuGraphs: A Performance Prediction Dataset on Large Tensor Computational Graphs*, NeurIPS 2023. arXiv:2308.13490.
- **GCN** — Kipf & Welling, *Semi-Supervised Classification with Graph Convolutional Networks*, ICLR 2017. arXiv:1609.02907.
- **GraphSAGE** — Hamilton, Ying & Leskovec, *Inductive Representation Learning on Large Graphs*, NeurIPS 2017. arXiv:1706.02216.
- **GAT** — Veličković et al., *Graph Attention Networks*, ICLR 2018. arXiv:1710.10903.
- **GPS** — Rampášek et al., *Recipe for a General, Powerful, Scalable Graph Transformer*, NeurIPS 2022. arXiv:2205.12454.
- **GNN expressivity / WL** — Xu et al., *How Powerful are Graph Neural Networks?* (GIN), ICLR 2019. arXiv:1810.00826.
- **ListMLE** — Xia et al., *Listwise Approach to Learning to Rank: Theory and Algorithm*, ICML 2008.
- **RankNet** — Burges et al., *Learning to Rank using Gradient Descent*, ICML 2005.
- **PyTorch Geometric** — Fey & Lenssen, docs at `pytorch-geometric.readthedocs.io`.

- **Stanford CS224W**, *Machine Learning with Graphs* — lecture course, excellent for GNN foundations.
- **Competition** — `kaggle.com/competitions/predict-ai-model-runtime`, and the official TF-GNN starter notebook in `notebooks/`.

10.3 Suggested study order (mapped to the workflow phases)

1. **Before Phase 1:** Sections 1, 3, 7 — understand the problem, graphs, and the dataset schema. You cannot write a correct loader without Section 7.
2. **Before Phase 2:** Sections 2 and 4 — ranking losses and metrics, plus NN basics, so the first GNN and its OPA evaluation make sense.
3. **During Phase 2–3:** Sections 5 and 6 — GNN architectures and, critically, Graph Segment Training for the layout collections.
4. **During Phase 4–5:** revisit Sections 5.4–5.8 and 8.6–8.7 — attention/GPS, expressivity and feature engineering, and ensembling, which is where competitive score is found.
5. **Throughout:** Section 9 — experimental discipline keeps the whole effort honest.